one variable sum in the preceding program. (You can override this behavior, as we'll see later.)

If, however, a variable appears only inside a block, then that variable is local to the block—in the preceding program, we couldn't have written the value of square in the puts statement at the end of the code, because square is no longer defined at that point. It is defined only inside the block itself.

This scoping behavior can lead to unexpected problems. For example, say our program was dealing with drawing different shapes. We might have this:

```
 # assume Shape is defined elsewhere
square = Shape.new(sides: 4)
 # .. lots of code
sum = 0

[1, 2, 3, 4].each do |value|
  square = value * value
  sum += square
end
puts sum

square.draw   # Error! Square is a number now...
```

This code would fail, because the variable square, which originally held a Shape object, will have been overwritten inside the block and will hold a number by the time the each method returns. This problem doesn't bite often, but when it does, it can be very confusing.

Fortunately, Ruby has a couple of answers.

Parameters to a block are *always* local to a block, even if they have the same name as variables in the surrounding scope. (You'll get a warning message when you do this if you run Ruby with the -w option.)

In this example, declaring thing as a parameter of the block, means that the block gets its own version of thing and the value outside the block is undisturbed by the rest of the block:

```
thing = "some shape"
[1, 2].each { |thing| puts thing }
puts thing
```

produces:

```
1
2
some shape
```

Second, you can define block-local variables by putting them after a semicolon in the block's parameter list. So, in our sum-of-squares example, we should have indicated that the square variable was block-local by writing it as follows:

```
square = "some shape"

sum = 0
[1, 2, 3, 4].each do |value; square|
  square = value * value    # this is a different variable
  sum += square
end
puts sum
```